
Blend Web Service

Technical Specification

Events & Ticketing API — Whish Integration

Version	Author	Date	Comment
1.0	Blend Engineering	2026-07-17	Initial release — Events catalog, Event details, Checkout (Whish Pay), Order status.

Disclaimer

The information in this document is provided for Whish as an authorized Blend integration partner. Blend retains the right to make changes to this specification at any time. Web service requests, responses, and error codes are subject to change. This document is provided for informational purposes only. The content of this document is the sole property of Blend.

Version Control

Overview

Environments

Communication Requirements

Request Headers

Response Structure

Integration Flow

List Events

Get Event Details

Checkout — Create Payment

Get Order Status

Overview

The Blend API is a REST-based service accessible over HTTPS. It supports JSON-formatted requests and responses and is intended to be consumed by authorized partners. It lets Whish list Blend's platform events, read event and ticket details, create a ticket order paid through Whish Pay, and read the resulting order status. Blend remains the source of truth for inventory, pricing, seating and ticket issuance; Whish presents the events and collects the payment.

Environments

Production

Base URL: `https://api.blendapp.ai/api/v1`

Sandbox

Base URL: `https://api-staging.blendapp.ai/api/v1`

Communication Requirements

Protocol and Format

- The partner must be capable of producing and consuming JSON over HTTPS.
- All requests and responses use the `application/json` format.

Request Headers

All requests sent to Blend services must include the following HTTP headers.

Header Name	Description
<code>x-blend-key</code>	Partner key — value provided by Blend. Identifies Whish and authorizes access to the platform catalog.
<code>x-blend-secret</code>	Partner secret — value provided by Blend. Paired with the key on every request.
<code>X-Forwarded-For</code>	The end buyer's device IP, so per-buyer rate limits apply to the buyer and not to Whish's backend.
<code>User-Agent</code>	Identifies the consuming application, e.g. <code>Whish/1.0</code> .
<code>Content-Type</code>	<code>application/json</code>

Keyed access. The key/secret pair authenticates Whish across the whole Blend platform (all organizers' public events). Access is rate-limited and can be revoked at any time. There is no anonymous access to the catalog.

Response Structure

All API responses follow a unified structure with the attributes listed below.

Attribute	Description
<code>success</code>	Boolean value indicating success or failure.
<code>code</code>	Operation-specific failure code (<code>null</code> on success).
<code>message</code>	Optional human-readable message (<code>null</code> on success).
<code>data</code>	Contains the response payload based on the executed operation.

Notes

- Missing or invalid headers result in request rejection with `401 MISSING_KEY` / `INVALID_KEY`.
- All monetary amounts are in major units (e.g. dollars, not cents). The Whish Pay leg is USD only.

Integration Flow

A ticket purchase inside the Whish app follows five steps. Steps 1–4 are calls Whish makes to Blend; step 5 (the payment settlement and confirmation) happens between Whish Pay and Blend server-to-server.

#	Step	Call / action
1	List events	<code>GET /channel/events</code> — Whish shows every Blend platform event as a card.
2	Open an event	<code>GET /channel/events/{id}</code> — tickets, waves, seating, custom questions.
3	Create the order	<code>POST /channel/events/{id}/checkout</code> — Blend prices the order, reserves it, and opens a Whish Pay collect session. Returns <code>collectUrl</code> + <code>externalId</code> + <code>orderId</code> .
4	Buyer pays	The buyer settles the collect session with their Whish balance inside the Whish app.
5	Confirm & issue	Whish Pay notifies Blend; Blend verifies the collect, books the tickets, and issues them. Whish reads <code>POST /channel/orders/status</code> until <code>paid</code> .

Pricing is server-authoritative. Blend computes the amount at checkout (ticket price + fees). The `amount` Whish sends must equal Blend's computed total ($\pm \$0.01$) or the request is rejected with `AMOUNT_MISMATCH`. Always show the buyer the amount Blend returns.

Money flow. The collect session is created on Blend's Whish Pay merchant account; the buyer's payment settles to Blend, and Blend reconciles and pays out the event organizer internally. Whish is not the merchant of record for the ticket.

List Events

Returns the paginated list of Blend platform events available for sale, across all organizers. Use it to render the events screen in the Whish app.

URI

GET /channel/events?page=1&limit=20

Request

No request body. Optional query parameters:

Parameter	Description	Type
page	Page number, starting at 1 (default 1).	Integer
limit	Items per page (default 20, max 50).	Integer
search	Free-text search over event title and venue.	String
city	Filter by city.	String
category	Filter by category.	String
from / to	ISO dates — only events within the range.	String

Response Data

Field	Description	Type
events[]	Array of event cards.	Array
events[].id	Event identifier (use in the detail + checkout calls).	String
events[].title	Event title.	String
events[].coverImageUrl	Card image URL.	String
events[].startDate	Start date/time (ISO 8601).	String
events[].timezone	Event timezone — render dates in this zone.	String
events[].venue	Venue / location.	String
events[].priceFrom	Lowest ticket price ("from" display).	Double
events[].currency	Currency code (e.g. USD).	String
events[].soldOut	Whether the event is fully sold out.	Boolean
page / total	Current page and total event count.	Integer

Response Example

```
{
  "success": true, "code": null, "message": null,
  "data": {
    "events": [
      {
        "id": "aBc123Xy",
        "title": "Imad Kehdi - 17",
        "coverImageUrl": "https://cdn.blendapp.ai/.../cover.jpg",
        "startDate": "2026-07-30T16:00:00.000Z",
        "timezone": "Asia/Beirut",
        "venue": "The Hidden Cellar, Achrafieh",
        "priceFrom": 25, "currency": "USD", "soldOut": false
      }
    ],
    "page": 1, "total": 42
  }
}
```

Get Event Details

Returns the full detail for a single event: description, tickets, waves, add-ons, custom questions and seating. Use it to build the event page and the ticket selector.

URI

GET /channel/events/{eventId}

Response Data (key fields)

Field	Description	Type
title, description	Event title and description.	String
startDate, endDate, timezone, venue	Schedule and location.	String
gallery[]	Image URLs.	Array
currency	Event currency.	String
tickets[].id	Ticket identifier — pass in checkout.	String
tickets[].name	Ticket tier name.	String
tickets[].price	Live price (wave / demand pricing applied).	Double
tickets[].currency	Currency.	String
tickets[].remaining / soldOut	Remaining count (<code>null</code> = untracked) and sold-out flag.	Int / Bool
tickets[].maxPerOrder	Max quantity per order.	Integer
tickets[].salesStartTime / salesEndTime / onSaleNow	On-sale window (ISO, nullable) and whether the tier is purchasable now.	String / Bool
customQuestions[]	Questions the buyer must answer: <code>{id, label, type, required, options}</code> .	Array
seating	<code>{ enabled, provider }</code> — when enabled, seats are chosen and held before checkout (see note).	Object
flags	<code>{ accessGated, waitingRoom, waitlist, requiresApproval }</code> — gates to honor before checkout.	Object

Response Example (abridged)

```
{
  "success": true, "data": {
    "id": "aBc123Xy", "title": "Imad Kehdi — 17",
    "startDate": "2026-07-30T16:00:00.000Z", "timezone": "Asia/Beirut",
    "currency": "USD",
    "tickets": [
      { "id": "665f00...", "name": "Back Area", "price": 25,
        "currency": "USD", "remaining": 120, "soldOut": false,
        "maxPerOrder": 10, "onSaleNow": true }
    ],
    "customQuestions": [], "seating": { "enabled": false, "provider": null },
    "flags": { "accessGated": false, "waitingRoom": false }
  }
}
```

Reserved seating. When `seating.enabled` is true, seats are selected and held (10-minute hold) before checkout, and the hold reference is passed to the checkout call. Seat-map details are covered in

the seating supplement.

Checkout — Create Payment

Creates the order for the selected tickets and opens a Whish Pay collect session on Blend's merchant account. Returns the collect URL that the Whish app settles, plus the identifiers used to confirm and read the order.

URI

POST /channel/events/{eventId}/checkout

Request

Besides the mandatory headers, the following attributes are attached in the request body:

Field	Description	Type
amount	Total to be paid — must equal Blend's computed total ($\pm$ \$0.01).	Double
currency	Must be <code>USD</code> .	String
buyerName	Buyer's full name.	String
buyerEmail	Buyer's email — ticket is delivered here.	String
buyerPhone	Buyer's phone (optional).	String
tickets[]	Line items: <code>[{ ticketId, quantity }]</code> .	Array
customQuestionResponses[]	Answers: <code>[{ questionId, answer }]</code> (when the event has questions).	Array
accessCode	Access code (only for access-gated events).	String
successCallbackUrl	Optional GET URL Blend calls on success (for Whish-side reconciliation).	String
failureCallbackUrl	Optional GET URL Blend calls on failure.	String
idempotencyKey	Unique per checkout attempt (UUID). Prevents duplicate orders on retry.	String

Request Sample

```
{
  "amount": 49.5, "currency": "USD",
  "buyerName": "Rami Khoury", "buyerEmail": "rami@example.com",
  "buyerPhone": "+96170123456",
  "tickets": [{ "ticketId": "665f00...", "quantity": 2 }],
  "idempotencyKey": "whish-7f3a9c1b24d8",
  "successCallbackUrl": "https://whish.money/api/blend/success",
  "failureCallbackUrl": "https://whish.money/api/blend/failure"
}
```

Response Data

Field	Description	Type
orderId	Blend order identifier (use in Get Order Status).	String
externalId	Whish Pay collect reference (numeric).	Long
collectUrl	Whish Pay collect page URL to settle the payment.	String
amount / currency	Confirmed total and currency.	Double / String

```
{
  "success": true, "data": {
    "orderId": "665fab...",
    "externalId": 1234567890123,
    "collectUrl": "https://whish.money/pay/8nQS2mL",
    "amount": 49.5, "currency": "USD"
  }
}
```

Failure Codes

AMOUNT_MISMATCH , SOLD_OUT , SALES_NOT_STARTED / SALES_ENDED , EVENT_CANCELLED ,
ACCESS_CODE_REQUIRED , SEAT_UNAVAILABLE , WHISH_DISABLED_FOR_EVENT .

Get Order Status

Returns the current status of a Blend order. Poll it after the buyer settles the collect (recommended every 2–3 seconds) until `paid` is `true`, then present the ticket.

URI

POST /channel/orders/status

Request

Field	Description	Type
<code>externalId</code>	The Whish Pay collect reference returned by checkout.	Long

Response Data

Field	Description	Type
<code>status</code>	Order status: <code>pending</code> , <code>paid</code> , or <code>failed</code> .	String
<code>paid</code>	Whether payment has settled and the ticket is issued.	Boolean
<code>ticket.shortId</code>	Ticket identifier.	String
<code>ticket.ticketUrl</code>	Ticket page URL (QR is shown here after buyer sign-in).	String
<code>ticket.attendeeName</code>	Attendee name.	String
<code>ticket.eventTitle</code>	Event title.	String
<code>ticket.seatLabels[]</code>	Reserved seats (if any).	Array

Response Example

```
{
  "success": true, "data": {
    "status": "paid", "paid": true,
    "ticket": {
      "shortId": "xYz98AbC",
      "ticketUrl": "https://tickets.blendapp.ai/ticket/xYz98AbC",
      "attendeeName": "Rami Khoury",
      "eventTitle": "Imad Kehdi – 17",
      "seatLabels": []
    }
  }
}
```

Ticket delivery. On payment, Blend also emails the buyer the ticket with a QR from Blend's own domain. The scan QR is shown on the ownership-gated ticket page after the buyer signs in with the purchase email.